4

MCP para ampliar tu agente de programación

Context7, Chrome DevTools, Supabase y Vercel: instalación paso a paso para Claude Code y Codex, pruebas de comprobación, ocho prompts y un ejercicio completo.

Fecha de verificación de la documentación: 20 de octubre de 2026.

Cómo usar esta guía

Esta guía te lleva de cero a cuatro conexiones funcionando y te propone un ejercicio completo para usarlas juntas. Para cada herramienta encontrarás los requisitos, los pasos para **Claude Code** y para **Codex**, una prueba sencilla, el resultado esperado y los errores frecuentes.

Antes de empezar, tres avisos:

- **Qué se verificó y qué no.** Todos los comandos y ajustes vienen de la documentación oficial consultada el 20 de octubre de 2026. **No se instaló ni se ejecutó nada**: no se afirma que los comandos se hayan probado, solo que coinciden con la documentación. Lo que no pudo verificarse está marcado como **«Pendiente de verificar»**.
- **Claude Code no es Claude Desktop ni Claude.ai.** Esta guía cubre Claude Code (la herramienta de terminal) y Codex (el agente de programación de OpenAI). Los conectores de Claude.ai o de Claude Desktop se configuran de otra forma y no se tratan aquí.
- **Instalar los cuatro MCP no garantiza código correcto ni aplicaciones listas para producción.** Amplían lo que el agente puede consultar y hacer; revisar y probar su trabajo sigue siendo tu responsabilidad.

Qué es MCP y cuándo conviene usarlo

MCP (Model Context Protocol, «protocolo de contexto de modelos») es un estándar abierto para que un agente de IA se conecte a herramientas y datos externos. Cada conexión se llama **servidor MCP** y ofrece al agente una lista de **herramientas** (acciones que puede ejecutar, como «consultar documentación» o «listar tablas»).

Qué añade al agente:

- **Información actual**, en lugar de depender solo de lo que aprendió en su entrenamiento (documentación de librerías).
- **Ojos y manos en el navegador**: abrir una página real, leer su consola y ver errores.
- **Acceso a datos** de tu base de datos, con los permisos que tú definas.
- **Acceso a tu plataforma de despliegue** para consultar estados y logs.

Cuándo conviene usarlo: cuando la tarea necesita información o acciones que el agente no tiene dentro del proyecto. Cuándo **no**: si el agente ya puede resolverlo leyendo archivos locales, o si no quieres que tenga acceso a esa cuenta o datos.

Servidores locales y remotos

- **Remoto:** el servidor corre en internet y tu agente se conecta por una dirección web (URL). Suele autenticarse con una cuenta, a través del navegador (OAuth). Ejemplos de esta guía: Context7 (también existe en modo local), Supabase y Vercel.
- **Local:** el servidor es un programa que corre en tu computador y el agente lo lanza (en esta guía, con `npx`, que ejecuta un paquete de Node.js sin instalarlo de forma permanente). Ejemplo: Chrome DevTools MCP, que controla un Chrome de tu equipo.

Seguridad básica (aplica a todos). Un servidor MCP actúa con tus permisos. Conecta solo servidores de confianza, deja activada la confirmación manual de las acciones del agente y desconfía del contenido externo: si un texto leído por el agente contiene instrucciones ocultas, puede intentar desviarlo (se llama *inyección de instrucciones*, en inglés *prompt injection*). Las documentaciones de Supabase y de Vercel insisten en esto.

Las cuatro herramientas de un vistazo



HERRAMIENTA	FUNCIÓN	EJEMPLO DE USO	TIPO	REQUISITOS (SEGÚN SU DOCUMENTACIÓN)
Context7	Trae documentación y ejemplos actualizados y específicos de la versión de una librería	«¿Cómo uso el enrutador de la versión de Next.js de mi proyecto?»	Remoto (URL) o local (`npx`)	Una clave de API es *recomendada* (gratuita en su panel) para límites mayores; el asistente de instalación `ctx7` pide Node.js 22.13 o superior
Chrome DevTools MCP	Controla e inspecciona un Chrome real: capturas, consola, red y rendimiento	«Abre mi página local y dime qué errores hay en la consola»	Local (`npx`)	Node.js (versión LTS), Chrome estable actual o más reciente, npm
Supabase MCP	Conecta el agente con tus proyectos de Supabase: tablas, consultas, registros, funciones	«Lista las tablas y propón un esquema de ejemplo»	Remoto (URL)	Cuenta de Supabase; inicio de sesión desde el navegador (OAuth)
Vercel MCP	Consulta proyectos, despliegues y registros de Vercel; según su documentación, también puede desplegar	«Revisa el último despliegue y los errores en tiempo de ejecución»	Remoto (URL)	Cuenta de Vercel y permisos sobre el equipo; inicio de sesión desde el navegador (OAuth)

Qué necesitas además, para todas: tener instalado Claude Code o Codex. Los comandos de Codex vienen de su documentación oficial y requieren la CLI de Codex.

Dónde se ejecuta cada comando

Para no confundirte, estas guías usan tres lugares:

- **Terminal:** la terminal normal de tu sistema (PowerShell, Terminal o Git Bash), **fuera** de Claude Code y de Codex. Aquí van los comandos que empiezan por `claude mcp ...`, `codex mcp ...` o `npx ...`.
- **Dentro de Claude Code:** la sesión interactiva que abres escribiendo `claude` en la terminal. Aquí van `/mcp` y tus prompts.
- **Dentro de Codex:** la interfaz que abres escribiendo `codex`. Aquí van `/mcp` y tus prompts.

Para abrir cada una: entra primero en la carpeta de tu proyecto con `cd ruta/de/tu-proyecto` y escribe `claude` o `codex`.

Cómo funciona la configuración en cada agente

Claude Code

- Los servidores se añaden con `claude mcp add`. Para uno remoto: `claude mcp add --transport http <nombre> <url>`. Para uno local: `claude mcp add <nombre> -- <comando> [argumentos]` (el `--` separa las opciones de Claude de las del servidor).
- Tiene tres alcances: **local** (por defecto: solo tú y solo este proyecto), **proyecto** (`--scope project`: se guarda en un archivo `.mcp.json` en la raíz y se comparte por control de versiones) y **usuario** (`--scope user`: todos tus proyectos, privado). Si el mismo servidor está en varios sitios, gana local, luego proyecto y luego usuario.
- Para ver y probar: `claude mcp list`, `claude mcp get <nombre>` y `claude mcp remove <nombre>`; dentro de Claude Code, `/mcp` muestra el estado y permite autenticar.
- Un servidor de `.mcp.json` pide tu aprobación antes de usarse en sesiones interactivas.

Codex

- Los servidores se guardan en `config.toml`: por defecto `~/.codex/config.toml`; un proyecto puede tener `.codex/config.toml`, pero solo se usa si confías en el proyecto. La aplicación de escritorio, la CLI y la extensión de IDE comparten esa configuración.
- Se añaden con `codex mcp add`: para uno local, `codex mcp add <nombre> -- <comando>`; para uno remoto, con `--url`. `codex mcp list` los muestra y `codex mcp login <nombre>` inicia sesión en los que usan OAuth. Dentro de Codex, `/mcp` lista los servidores activos.
- Cada servidor es una tabla `[mcp_servers.<nombre>]` con campos como `command` y `args` (local) o `url` (remoto). La documentación indica `startup_timeout_sec` (por defecto 10 segundos) para el tiempo de arranque.

Pendiente de verificar: la documentación de Codex consultada no describe `codex mcp get` ni `codex mcp remove` ni notas específicas de Windows para MCP; la de Claude Code consultada tampoco incluye en la parte revisada una nota de Windows sobre `npx`. Consulta tu versión con `codex mcp --help` y la documentación oficial.

1. Context7: documentación actualizada de las librerías de tu proyecto

Qué es. Context7 es un servidor que lleva documentación y ejemplos de código actualizados y de la versión que mencionas al contexto del agente, para que dependa menos de información antigua. Documentación oficial: github.com/upstash/context7.

Requisitos previos

- Claude Code o Codex instalados.
- Para el modo remoto: acceso a internet y, según su documentación, una clave de API recomendada (gratuita en context7.com/dashboard) para obtener límites mayores. La documentación consultada no da cifras de límites ni indica si se puede usar sin clave: **pendiente de verificar**.
- Para el modo local con `npx`: Node.js. Para el asistente `ctx7`: Node.js 22.13 o superior.

Opción rápida (según su documentación): en la terminal, `npx ctx7 setup` te autentica con OAuth, genera una clave de API e instala una habilidad (*skill*). Con `--claude` apunta a Claude Code. Para quitarlo: `npx ctx7 remove`. Esta guía no detalla ese asistente; abajo están los pasos manuales documentados.

Pasos para Claude Code

Paso 1. Terminal. Entra en tu proyecto: `cd ruta/de/tu-proyecto`.

Paso 2. Terminal. Añade el servidor remoto (sustituye `TU_CLAVE_API`):

```
claude mcp add --scope user --transport http context7 https://mcp.context7.com/mcp --header "Authorization: Bearer TU_CLAVE_API"
```

Alternativa local (también de la documentación oficial de Context7):

```
claude mcp add --scope user context7 -- npx -y @upstash/context7-mcp --api-key TU_CLAVE_API
```

Paso 3. Terminal. Comprueba que quedó registrado: `claude mcp list`.

Paso 4. Terminal. Abre Claude Code con `claude`.

Paso 5. Dentro de Claude Code. Escribe `/mcp` y comprueba que `context7` aparece conectado.

Pasos para Codex

Paso 1. Terminal. Añade el servidor local con el comando que aparece en la documentación de Codex:

```
codex mcp add context7 -- npx -y @upstash/context7-mcp
```

Para pasar la clave, la documentación de Context7 muestra esta variante:

```
codex mcp add context7 -- npx -y @upstash/context7-mcp --api-key TU_CLAVE_API
```

Paso 2. Alternativa remota, en `~/.codex/config.toml` (o `.codex/config.toml` en un proyecto de confianza):

```
[mcp_servers.context7]
url = "https://mcp.context7.com/mcp"
http_headers = { "Authorization" = "Bearer TU_CLAVE_API" }
```

Paso 3. Terminal. Comprueba con `codex mcp list`.

Paso 4. Dentro de Codex. Abre con `codex` y escribe `/mcp`.

Pendiente de verificar: la documentación de Context7 usa `startup_timeout_ms` en su ejemplo local de `config.toml`, mientras que la de Codex documenta `startup_timeout_sec`. Si ves errores de arranque, usa el campo documentado por Codex (`startup_timeout_sec`) y consulta ambas páginas.

Autenticación. Con clave: encabezado `Authorization: Bearer TU_CLAVE_API`. La documentación indica también que, para usar OAuth en lugar de una clave, la dirección cambia de `/mcp` a `/mcp/oauth` (solo para conexiones remotas por HTTP).

Prueba sencilla. En el agente, pide: `Usa Context7 para explicar cómo definir una ruta con la versión de [LIBRERÍA] que usa este proyecto. Cita la versión.` La documentación indica que basta con añadir «use context7» al prompt, y que puedes indicar directamente un identificador de librería con la sintaxis `/organizacion/proyecto` (por ejemplo, `use library /supabase/supabase`).

Resultado esperado. El agente llama a las herramientas de Context7 (según su documentación, `resolve-library-id` para encontrar la librería y `query-docs` para consultarla) y responde con ejemplos acordes a la versión que mencionaste.

Errores frecuentes

- No escribir la clave o dejar `TU_CLAVE_API` sin sustituir.
- No mencionar la versión: el agente puede traer ejemplos de otra versión. Indica la versión del proyecto (míralo en `package.json`).
- Confundir el servidor local con el remoto: el local necesita Node.js.
- Esperar que traiga documentación de librerías que Context7 no tiene indexadas.

2. Chrome DevTools MCP: tu agente mira una página real

Qué es. Es un servidor local que permite al agente controlar e inspeccionar un Chrome real: capturas, mensajes de la consola, solicitudes de red y trazas de rendimiento. Documentación oficial: github.com/ChromeDevTools/chrome-devtools-mcp.

Requisitos previos

- Node.js (versión LTS) y npm.
- Google Chrome estable actual o más reciente. El proyecto indica que da soporte oficial a Google Chrome y Chrome for Testing; otros navegadores basados en Chromium pueden funcionar, sin garantías.
- El navegador se inicia solo cuando una herramienta lo necesita por primera vez.

Privacidad y seguridad (según su documentación)

- La herramienta **expone el contenido del navegador** al cliente MCP: el agente puede inspeccionar y modificar datos del navegador o de las DevTools. **No compartas información sensible** y no abras páginas con cuentas personales o sesiones abiertas.
- Las estadísticas de uso están activadas por defecto; se desactivan con `--no-usage-statistics`. Las herramientas de rendimiento pueden enviar las direcciones de las trazas a la API CrUX de Google; se desactiva con `--no-performance-crux`.

Pasos para Claude Code

Paso 1. Terminal. Ejecuta el comando de la documentación del proyecto:

```
claude mcp add chrome-devtools --scope user npx chrome-devtools-mcp@latest
```

Paso 2. Terminal. Comprueba con `claude mcp list`.

Paso 3. Terminal. Abre `claude` en tu proyecto.

Paso 4. Dentro de Claude Code. Escribe `/mcp` y comprueba que `chrome-devtools` está conectado.

Nota: la sintaxis general de Claude Code separa las opciones del comando del servidor con `--`. El comando de arriba es el que muestra el proyecto; si tu versión lo rechaza, consulta `claude mcp add --help`.

Pasos para Codex

Paso 1. Terminal. Ejecuta el comando de la documentación del proyecto:

```
codex mcp add chrome-devtools -- npx chrome-devtools-mcp@latest
```

Paso 2. Terminal. Comprueba con `codex mcp list`.

Paso 3. Dentro de Codex. Abre `codex` y escribe `/mcp`.

Configuración alternativa en `config.toml` para Windows 11 (según la documentación del proyecto; ajusta rutas si tu instalación es distinta):

```
[mcp_servers.chrome-devtools]
command = "cmd"
args = [
    "/c",
    "npx",
    "-y",
    "chrome-devtools-mcp@latest",
]
env = { SystemRoot="C:\\Windows", PROGRAMFILES="C:\\Program Files" }
startup_timeout_ms = 20_000
```

Pendiente de verificar: esa documentación usa `startup_timeout_ms`; la de Codex documenta `startup_timeout_sec`. Si el servidor tarda en arrancar, usa el campo que documente tu versión de Codex.

Configuración estándar en formato JSON (clientes que usan `mcpServers`, según el proyecto):

```
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest"]
    }
  }
}
```

Modo ligero para tareas básicas: añade `--slim` y `--headless` a los argumentos. Para conectarte a un Chrome que ya esté abierto con depuración remota, el proyecto documenta `--browser-url` (por ejemplo, `--browser-url=http://127.0.0.1:9222`).

Autenticación. No requiere cuenta ni clave.

Prueba sencilla. El proyecto propone: «Check the performance of https://developers.chrome.com». En español: `Comprueba el rendimiento de https://developers.chrome.com`. Para tu proyecto: `Abre http://localhost:3000, toma una captura y dime si hay errores en la consola.`

Resultado esperado. Se abre una ventana de Chrome controlada por el agente (o corre sin ventana si usas `--headless`), el agente navega a la dirección y responde con lo que encontró: capturas, mensajes de consola o resumen de rendimiento. Según la documentación, entre las herramientas hay `navigate_page`, `take_screenshot`, `list_console_messages`, `list_network_requests` y `performance_start_trace`.

Errores frecuentes

- No tener Chrome instalado o tener una versión antigua.
- El servidor tarda en arrancar la primera vez (descarga el paquete): reintenta o aumenta el tiempo de arranque.
- Abrir páginas con sesiones personales abiertas y dejar que el agente las inspeccione.
- Pedir una prueba de una dirección local sin tener el servidor del proyecto en marcha.

3. Supabase MCP: tu base de datos al alcance del agente (con cuidado)

Qué es. El servidor MCP de Supabase conecta el agente con tus proyectos para consultar y gestionar tablas, ejecutar SQL, revisar registros y más. Documentación oficial: supabase.com/docs/guides/ai-tools/mcp.

Requisitos previos

- Una cuenta de Supabase y, de preferencia, un **proyecto de pruebas** (no de producción).
- Claude Code o Codex instalados.
- Servidor remoto: `https://mcp.supabase.com/mcp`. Para desarrollo local con la CLI de Supabase, la documentación indica `http://localhost:54321/mcp`.

Alcance y modo de solo lectura (parámetros de la dirección, según la documentación)

PARÁMETRO	QUÉ HACE
`project_ref=<ID>`	Limita el servidor a un solo proyecto y desactiva las herramientas de cuenta
`read_only=true`	Ejecuta las consultas con un usuario de Postgres de solo lectura
`features=<grupos>`	Activa solo los grupos de herramientas que indiques, separados por comas

Ejemplo de dirección: `https://mcp.supabase.com/mcp?project_ref=ID_DE_TU_PROYECTO&read_only=true`. El ID de tu proyecto lo ves en el panel de Supabase.

Autenticación. Por defecto usa OAuth: tu cliente abre el navegador, inicias sesión y eliges la organización; la documentación indica que no necesitas un token de acceso personal. Para integración continua (CI) existe la opción de un token con alcance limitado en un encabezado `Authorization`.

Riesgos y recomendaciones (según su documentación)

- Conectar un modelo a tus proyectos conlleva riesgos de seguridad; el principal es la inyección de instrucciones desde datos no confiables.
- Mantén la **aprobación manual** de las llamadas a herramientas.
- **Evita datos de producción** si la tarea no los necesita; usa alcance por proyecto, modo de solo lectura y grupos de herramientas restringidos.
- No des este servidor a clientes ni usuarios finales: corre con tus permisos de desarrollador.
- Usa ramas (*branching*) para probar cambios antes de aplicarlos; según la documentación, es una función experimental de un plan de pago.
- **Usa datos de ejemplo** (ficticios), como en el ejercicio de esta guía.

Pasos para Claude Code

Paso 1. Terminal. Entra en tu proyecto y ejecuta (el comando de la documentación usa el alcance de proyecto):

```
claude mcp add --scope project --transport http supabase "https://mcp.supabase.com/mcp?
project_ref=ID_DE_TU_PROYECTO&read_only=true"
```

Esto crea o actualiza un archivo `.mcp.json` en tu proyecto. **No incluyas claves en ese archivo**: se comparte con tu equipo.

Paso 2. Terminal. Abre `claude`.

Paso 3. Dentro de Claude Code. Escribe `/mcp`, selecciona `supabase` y elige «Authenticate» para iniciar sesión en el navegador. (La documentación de Supabase indica hacerlo en una terminal normal, no en la extensión del IDE.)

Pasos para Codex

Paso 1. Terminal. Añade el servidor:

```
codex mcp add supabase --url "https://mcp.supabase.com/mcp?project_ref=ID_DE_TU_PROYECTO&read_only=true"
```

O en `~/.codex/config.toml`:

```
[mcp_servers.supabase]
url = "https://mcp.supabase.com/mcp?project_ref=ID_DE_TU_PROYECTO&read_only=true"
```

Paso 2. Terminal. Inicia sesión: `codex mcp login supabase`.

Paso 3. Dentro de Codex. Abre `codex` y escribe `/mcp` para verificar.

Prueba sencilla. `Lista las tablas de mi proyecto de Supabase y dime cuántas filas tiene cada una. No modifiques nada.`

Resultado esperado. El agente usa una herramienta de listado de tablas y responde con la lista (puede estar vacía si el proyecto es nuevo). Si activaste `read_only=true`, cualquier intento de escribir debería fallar.

Errores frecuentes

- Conectar el proyecto de producción con permisos de escritura.
- Olvidar autenticar: el servidor aparece «necesita autenticación» hasta que inicias sesión.
- Escribir claves en `.mcp.json` y subirlo al repositorio.
- Usar `project_ref` con el ID equivocado y no ver tus tablas.
- Aprobar acciones sin leerlas.

Pendiente de verificar: qué planes de Supabase incluyen cada herramienta (la documentación consultada solo menciona un plan de pago para las ramas experimentales).

4. Vercel MCP: consultar despliegues no es lo mismo que crearlos

Qué es. Es el servidor MCP oficial de Vercel, remoto y con OAuth, en `https://mcp.vercel.com`. Según su documentación, sirve para buscar en la documentación, gestionar proyectos y despliegues, analizar logs y consultar analítica web. Está disponible en todos los planes y su uso está sujeto a los términos de productos de IA de Vercel. Documentación oficial: vercel.com/docs/agent-resources/vercel-mcp.

Requisitos previos

- Una cuenta de Vercel y los permisos necesarios sobre el equipo y el proyecto.
- Claude Code o Codex instalados. La documentación de Vercel indica que solo admite clientes revisados y aprobados por Vercel; ambos aparecen en su lista.
- Navegador para iniciar sesión (OAuth).

Consultar frente a crear

- **Consultar (leer):** herramientas como `list_deployments` (listar despliegues), `get_deployment` (ver estado y detalles), `get_runtime_logs` y `get_runtime_errors` (ver registros y errores), `list_projects`, `list_teams` y `get_project`. Son seguras para empezar.
- **Crear o cambiar:** según un anuncio de Vercel del 23 de julio de 2026, el servidor puede desplegar código a un proyecto nuevo o existente con la herramienta `deploy_to_vercel`. Hay también herramientas que modifican recursos (variables de entorno, dominios…) en su catálogo. **Pendiente de verificar:** el anuncio no especifica requisitos de Git ni de la CLI, ni si el despliegue es de vista previa o de producción. Antes de pedir un despliegue, comprueba tu versión de la herramienta en la lista de tu conexión.

Cuándo intervienen permisos, repositorio o CLI

- **Permisos:** según la documentación, añadir el servidor **no completa la autorización**. Debes iniciar sesión y conceder acceso a los equipos con los que quieres trabajar; tus permisos de Vercel siguen aplicando. Para comprobarlo, pide al asistente que liste tus equipos y proyectos.
- **Repositorio:** la documentación de MCP no lo exige para consultar. Para el despliegue de vista previa del ejercicio, la vía más habitual es conectar el repositorio de Git en Vercel; esta guía no verificó los pasos de esa integración: **pendiente de verificar** en la documentación de Vercel.
- **CLI:** MCP y la CLI de Vercel usan **autenticaciones separadas**: que la CLI funcione no prueba que tu conexión MCP tenga el mismo acceso. Si una acción no está disponible por MCP, la documentación remite al panel o a la CLI.

Seguridad (según su documentación). Conectar Vercel MCP da al sistema de IA el mismo acceso que tiene tu usuario de Vercel. Verifica que la dirección sea `https://mcp.vercel.com`, usa solo clientes de confianza, desconfía de instalaciones de un clic en mercados de terceros y **mantén la confirmación humana** de cada acción.

Pasos para Claude Code

Paso 1. Terminal. Entra en tu proyecto: `cd ruta/de/tu-proyecto`.

Paso 2. Terminal. Añade el servidor (comando de la documentación de Vercel):

```
claude mcp add --transport http vercel https://mcp.vercel.com
```

Paso 3. Terminal. Abre `claude`.

Paso 4. Dentro de Claude Code. Escribe `/mcp`, selecciona `vercel` y autentica en el navegador. Concede acceso al equipo que corresponda.

Pasos para Codex

Paso 1. Terminal. Añade el servidor (comando de la documentación de Vercel):

```
codex mcp add vercel --url https://mcp.vercel.com
```

Según la documentación, Codex detecta el soporte de OAuth y abre el navegador para autorizar la conexión. Si no ocurre, usa `codex mcp login vercel`.

Paso 2. Dentro de Codex. Abre `codex` y escribe `/mcp` para verificar.

Instalación para varios agentes a la vez. Vercel documenta una utilidad (`npx add-mcp https://mcp.vercel.com`) que detecta tus clientes de IA y los configura. No es necesaria si usas los comandos de arriba.

Prueba sencilla. `Lista mis equipos y proyectos de Vercel. No hagas ningún cambio.`

Resultado esperado. El agente responde con tus equipos y proyectos. Si falta un equipo, revisa que hayas iniciado sesión con la cuenta correcta y concedido acceso a ese equipo.

Errores frecuentes

- Añadir el servidor y olvidar autenticar y conceder acceso al equipo.
- Pedir un despliegue sin revisar el proyecto ni el equipo destino.
- Suponer que la CLI y MCP comparten sesión.
- Dar al agente acceso a un equipo de producción para una prueba.

Ocho prompts reutilizables

Sustituye los campos entre corchetes. El archivo `prompts-4-mcp.md` contiene estos mismos prompts.

Context7

1. Documentación para mi versión

```
Usa Context7 para consultar la documentación de [LIBRERÍA] en la versión [VERSIÓN] que usa este proyecto (míralo en package.json). Explícame cómo [TAREA]. Cita la versión que consultaste y no uses ejemplos de otras versiones. Si Context7 no tiene esa versión, dímelo.
```

2. Revisar código con documentación actual

```
Revisa [ARCHIVO] y, usando Context7 con la versión de [LIBRERÍA] de este proyecto, indica si hay funciones o patrones obsoletos. Para cada hallazgo muestra la línea, qué dice la documentación actual y una propuesta. No modifiques nada hasta que yo lo apruebe.
```

Chrome DevTools MCP

3. Auditar una página real

```
Abre [URL DE MI PÁGINA LOCAL] con Chrome DevTools. Toma una captura, revisa los mensajes de la consola y las solicitudes de red que fallan, y dame un informe con: errores, solicitudes lentas y qué archivo probablemente los causa. No abras sesiones personales ni inicies sesión en ninguna cuenta.
```

4. Probar un flujo

```
Con Chrome DevTools, prueba este flujo en [URL]: [PASOS DEL FLUJO]. Después de cada paso, indica qué viste y toma una captura si algo falla. Al final dime qué pasos funcionaron, cuáles no y qué cambiarías en el código. No envíes formularios con datos reales.
```

Supabase MCP

5. Explorar el esquema en modo lectura

```
Usando Supabase MCP en modo de solo lectura, lista las tablas de mi proyecto de pruebas y describe sus columnas y relaciones. Propón mejoras de diseño, pero no ejecutes ninguna instrucción que cambie datos o estructura.
```

6. Datos de ejemplo

```
En mi proyecto de pruebas de Supabase, crea una tabla [NOMBRE] con las columnas [COLUMNAS] y añade [NÚMERO] filas de datos de ejemplo ficticios. Antes de ejecutar, muéstrame el SQL y espera mi aprobación. No toques otras tablas. Todos los datos deben ser inventados.
```

Vercel MCP

7. Diagnosticar un despliegue (solo lectura)

```
Con Vercel MCP, busca el último despliegue del proyecto [PROYECTO] del equipo [EQUIPO]. Dime su estado, y si falló, revisa los registros y errores y explica la causa probable. Solo consulta: no cambies variables de entorno, dominios ni despliegues.
```

8. Preparar una vista previa

```
Quiero una vista previa del proyecto [PROYECTO]. Primero dime qué opciones hay para desplegar una vista previa con mis herramientas (conexión con Git, la CLI de Vercel o Vercel MCP) y qué requiere cada una. No despliegues nada todavía: espera a que yo elija y apruebe.
```

Ejercicio completo: una pequeña aplicación de recursos

Objetivo. Construir una aplicación sencilla que muestre una lista de recursos (título, enlace y etiqueta), usando la documentación actualizada, probándola en el navegador, con datos de ejemplo en Supabase y con una vista previa en Vercel. **Todos los datos son ficticios.** No es una aplicación lista para producción.

Antes de empezar

- Crea una carpeta nueva y vacía para el ejercicio (no uses un proyecto real).
- Usa un proyecto de **pruebas** de Supabase y un equipo o proyecto de pruebas de Vercel.
- Mantén activada la aprobación manual de cada acción del agente.

Pasos

1. **Conecta las cuatro herramientas** siguiendo las secciones 1 a 4 y comprueba cada una con su prueba sencilla.
2. **Documentación (Context7).** Pide con el prompt 1: «Usa Context7 para explicar cómo crear una aplicación web sencilla con [FRAMEWORK] en la versión [VERSIÓN]». Elige tú el framework; esta guía no impone ninguno.
3. **Base de datos (Supabase, solo lectura primero).** Con el prompt 5, revisa que el proyecto esté vacío o listo.
4. **Datos de ejemplo (Supabase).** Con el prompt 6, crea la tabla `recursos` con las columnas `titulo`, `enlace` y `etiqueta`, y cinco filas ficticias (por ejemplo, «Guía de ejemplo 1», `https://example.com/1`, «ejemplo»). Revisa el SQL antes de aprobarlo.
5. **Construye la aplicación.** Pide al agente: «Crea una página que lea la tabla `recursos` de Supabase y muestre la lista. Usa la documentación que consultaste. No pongas claves en el código: usa variables de entorno y dime cuáles». Las claves y direcciones del proyecto las obtienes del panel de Supabase; no las pegues en el chat.
6. **Prueba en el navegador (Chrome DevTools).** Arranca tu servidor local y usa el prompt 3 con tu dirección local. Corrige los errores de consola que aparezcan.
7. **Vista previa (Vercel).** Con el prompt 8, decide cómo desplegarás. Antes de nada comprueba con el prompt 7 que el servidor ve tu proyecto de Vercel. Desplegar una vista previa puede requerir conectar el repositorio de Git en Vercel o usar la CLI: consulta la documentación de Vercel (pendiente de verificar en esta guía).
8. **Revisa el resultado.** Abre la dirección de la vista previa, comprueba que la lista se ve y que la consola no muestra errores. Anota lo que no funcionó.
9. **Limpia.** Borra el proyecto de pruebas de Supabase y el de Vercel si ya no los necesitas, y retira los servidores que no quieras mantener (`claude mcp remove <nombre>`).

Qué demuestra y qué no. Si todo sale bien, habrás comprobado que cada herramienta aporta algo distinto. No demuestra que el código sea seguro, accesible ni listo para producción: revisa permisos de la base de datos, manejo de errores y pruebas antes de publicar nada real.

Checklist de instalación y comprobación

El archivo `checklist-4-mcp.md` contiene este mismo checklist.

Antes de empezar

- [ ] Tengo Claude Code o Codex instalado y sé en qué carpeta voy a trabajar.
- [ ] Tengo Node.js (versión LTS; Context7 con el asistente `ctx7` pide 22.13 o superior) y npm.
- [ ] Tengo Chrome estable y actualizado.
- [ ] Tengo cuenta de Supabase y de Vercel, de preferencia con proyectos de pruebas.
- [ ] Sé dónde ejecutar cada comando: terminal, dentro de Claude Code o dentro de Codex.

Context7

- [ ] Registré el servidor y aparece en `claude mcp list` o `codex mcp list`.
- [ ] La clave de API no está escrita en archivos del proyecto ni en el repositorio.
- [ ] La prueba cita la versión correcta de la librería.

Chrome DevTools MCP

- [ ] Registré el servidor y aparece en `/mcp`.
- [ ] La prueba abre una página y devuelve una captura o mensajes de consola.
- [ ] No abrí páginas con sesiones personales abiertas.

Supabase MCP

- [ ] Usé un proyecto de pruebas, con `project_ref` y `read_only=true`.
- [ ] Autentiqué desde el navegador y la lista de tablas se muestra.
- [ ] Mantengo la aprobación manual y no puse claves en `.mcp.json`.

Vercel MCP

- [ ] Autentiqué y concedí acceso solo al equipo necesario.
- [ ] La prueba lista mis equipos y proyectos.
- [ ] Sé qué herramientas solo consultan y cuáles pueden crear o cambiar cosas.

Al terminar

- [ ] Revisé el código y las pruebas antes de publicar nada real.
- [ ] Retiré los servidores y los proyectos de prueba que ya no necesito.

Glosario breve

- **Agente de programación:** programa con IA que lee tu código, ejecuta comandos y propone o hace cambios (por ejemplo, Claude Code o Codex).
- **MCP (Model Context Protocol):** estándar abierto para conectar agentes con herramientas y datos externos.
- **Servidor MCP:** una conexión concreta (documentación, navegador, base de datos…) que ofrece herramientas al agente.
- **Remoto / local:** un servidor remoto corre en internet y se usa por URL; uno local corre en tu equipo.
- **OAuth:** forma de iniciar sesión desde el navegador y autorizar a una aplicación sin darle tu contraseña.
- `npx`**:** comando de Node.js que ejecuta un paquete sin instalarlo de forma permanente.
- **Alcance (scope):** dónde se guarda una configuración: solo tú y este proyecto, todo el proyecto o todos tus proyectos.
- **Solo lectura:** modo en el que se pueden consultar datos pero no cambiarlos.
- **Vista previa (preview):** versión desplegada de prueba, distinta de la de producción.
- **Inyección de instrucciones (prompt injection):** texto externo que intenta darle órdenes ocultas al agente.
- **Variable de entorno:** valor que guardas fuera del código (por ejemplo, una clave) para no escribirlo en los archivos.

Enlaces oficiales y fecha de verificación

Fecha de verificación de la documentación: **20 de octubre de 2026**. Los comandos y las opciones pueden cambiar con las versiones: ante cualquier diferencia, manda la documentación oficial.

- Context7: github.com/upstash/context7 · panel de claves · instalación por cliente
- Chrome DevTools MCP: github.com/ChromeDevTools/chrome-devtools-mcp · configuración por cliente · referencia de herramientas
- Supabase MCP: supabase.com/docs/guides/ai-tools/mcp
- Vercel MCP: vercel.com/docs/agent-resources/vercel-mcp · herramientas · anuncio del despliegue por MCP
- Claude Code: documentación de MCP
- Codex: documentación de MCP de OpenAI (la dirección redirige a una página de documentación de OpenAI en `learn.chatgpt.com`)
- Estándar: modelcontextprotocol.io

Qué quedó pendiente de verificar

- Si Context7 puede usarse sin clave de API y sus límites de uso.
- El campo de tiempo de arranque de Codex: la documentación de Codex indica `startup_timeout_sec`; las de Context7 y Chrome DevTools MCP muestran `startup_timeout_ms`.
- Comandos `codex mcp get` y `codex mcp remove`, y notas de Windows para MCP en Claude Code y Codex.
- Requisitos de Git, de la CLI y tipo de despliegue (vista previa o producción) de la herramienta de despliegue de Vercel MCP, y los pasos de la integración con Git o de la CLI de Vercel para el ejercicio.
- Qué planes de Supabase incluyen cada herramienta.
- Que ninguno de los comandos se ejecutó en esta guía: se verificaron solo en la documentación.